

Cell 7 — TF-IDF + Logistic Regression + Deep NN (Notebook cell index: 14)

Grid search over TF-IDF $\times$ logistic regression, then TF-IDF dense features $\times$ MLP, per preprocessing variant.

Granular line-by-line breakdown

build_tfidf_dnn

```
def build_tfidf_dnn(input_dim, num_classes, dense1=256, dense2=128, drop=0
```

- `input_dim` — Number of TF-IDF features (vocabulary dimension after vectorization).
- `num_classes` — Output softmax size (topic count).
- `dense1, dense2, drop, lr` — Defaults for hidden sizes, dropout rate, Adam learning rate.

```
    model = Sequential([
        Dense(dense1, activation='relu', input_shape=(input_dim,)),
```

- `Sequential([...])` — List of layers stacked top-to-bottom.
- `Dense(dense1, ...)` — Fully connected layer with `dense1` neurons.
- `activation='relu'` — Rectified linear unit nonlinearity.
- `input_shape=(input_dim,)` — Only first layer needs explicit input shape; tuple length 1 for flat vectors.

```
        Dropout(drop),
        Dense(dense2, activation='relu'),
        Dropout(drop),
        Dense(64, activation='relu'),
        Dense(num_classes, activation='softmax')
    ])
```

- `Dropout(drop)` — Randomly zeros fraction `drop` of inputs during training (regularization).
- **Final** `Dense(num_classes, softmax)` — Outputs **probability vector** over classes.

```
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
```

- `Adam(learning_rate=lr)` — Adaptive optimizer instance.
- `loss='categorical_crossentropy'` — Matches **one-hot** targets `y_*_oh`.
- `metrics=['accuracy']` — Tracks batch/epoch accuracy during training.

```
    return model
```

- Caller receives compiled Keras model.
-

Hyperparameter grids (dicts)

```
tfidf_grid = {
    'max_features': [5000, 8000],
    'ngram_range': [(1, 1), (1, 2)],
    'min_df': [2]
}
```

- **Keys** — Must match `TfidfVectorizer` parameter names.
- **max_features** — Cap vocabulary at top-N frequent terms.
- **ngram_range** — (1,1) unigrams only; (1,2) unigrams + bigrams.
- **min_df** — Ignore terms appearing in fewer than 2 documents.

```
lr_grid = {
    'C': [0.5, 1.0, 2.0],
    'solver': ['lbfgs'],
    'max_iter': [400]
}
```

- **C** — Inverse regularization strength for `LogisticRegression`.
- **solver** — Optimization algorithm (lbfgs supports multiclass).
- **max_iter** — Upper bound on iterations per fit.

```
dnn_grid = {
    'dense1': [256, 384],
    'dense2': [128, 192],
    'drop': [0.35, 0.5],
    'lr': [1e-3],
    'batch_size': [64],
    'epochs': [50]
}
```

- Cartesian product of these keys drives DNN runs.

Outer loop over preprocessing

```
for prep_name, prep_fn in PREPROCESSORS.items():
```

- `.items()` — Yields pairs ('none', preprocess_none), etc.
- **prep_name** — String for logging.
- **prep_fn** — Callable preprocessor.

```
    print('\n' + '=' * 95)
    print('TF-IDF experiments for preprocessing:', prep_name)
```

- `'=' * 95` — Visual separator line.
- `print(..., prep_name)` — Prints which branch is running.

```
    X_train_text, X_val_text, X_test_text = get_preprocessed_splits(prep_f
```

- **Unpacking** — Three arrays of cleaned strings for train/val/test.

```
    best_lr_f1 = -1
    best_lr_model = None
    best_vectorizer = None
```

- **Sentinels** — Track best validation macro-F1 so far (-1 ensures first score wins) and the winning `LogisticRegression` + `TfidfVectorizer` objects.

TF-IDF + logistic regression grid

```
    for tfcfg in ParameterGrid(tfidf_grid):
```

- **ParameterGrid** — Iterator over every combination of keys in `tfidf_grid`.

```
        vectorizer = TfidfVectorizer(**tfcfg)
```

- ****tfcfg** — Unpacks dict as keyword args (max_features=5000, ...).

```
X_tr = vectorizer.fit_transform(X_train_text)
X_va = vectorizer.transform(X_val_text)
```

- **fit_transform on train** — Learns vocabulary + IDF from **training** headlines only.
- **transform on val** — Applies **same** vocabulary to validation text (no leakage).

```
for lrcfg in ParameterGrid(lr_grid):
```

- Nested loop over logistic regression hyperparameters.

```
lr_model = LogisticRegression(random_state=SEED, **lrcfg)
```

- ****lrcfg** — Expands C, solver, max_iter into constructor kwargs.

```
lr_model.fit(X_tr, y_train)
```

- **X_tr** — Sparse TF-IDF matrix; **y_train** — integer labels for **train_part**.

```
val_pred = lr_model.predict(X_va)
val_f1 = f1_score(y_val, val_pred, average='macro')
```

- **predict** — Integer class predictions on validation features.
- **f1_score** — Scalar validation metric for model selection.

```
add_tuning_log(prepare_name, 'TF-IDF', 'LogisticRegression', {**t
```

- **{**tfcfg, **lrcfg}** — Merges two dicts (TF-IDF + LR params) for logging.

```
print(f'LR cfg: {tfcfg} + {lrcfg} -> val_macro_f1={val_f1:.4f}')
if val_f1 > best_lr_f1:
    best_lr_f1 = val_f1
    best_lr_model = lr_model
    best_vectorizer = vectorizer
```

- **if** — Updates best model **and** its paired **vectorizer** (must stay consistent).

Evaluate best LR on test

```
X_te = best_vectorizer.transform(X_test_text)
test_pred_lr = best_lr_model.predict(X_te)
evaluate_and_store(y_test, test_pred_lr, prepare_name, 'TF-IDF', 'Logisti
```

- **transform** — Test headlines through **winning** TF-IDF.
- **predict** — Test predictions.
- **evaluate_and_store** — Prints metrics, plots CM, appends **all_results**.

Dense TF-IDF for DNN

```
X_tr_dense = best_vectorizer.fit_transform(X_train_text).toarray().astype(np.float32)
X_va_dense = best_vectorizer.transform(X_val_text).toarray().astype(np.float32)
X_te_dense = best_vectorizer.transform(X_test_text).toarray().astype(np.float32)
```

- **fit_transform on train again** — Refits TF-IDF with **same** class/settings as **best_vectorizer** on **same** grid hyperparameters (fresh matrix).
- **.toarray()** — Converts sparse matrix to dense NumPy (needed for **Dense** layers).
- **.astype(np.float32)** — Uses 32-bit floats (half memory vs float64; GPU-friendly).

DNN grid

```
best_dnn_f1 = -1
best_dnn_model = None
early_stop = EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True)
```

- **EarlyStopping** — Monitors **val_loss**; stops if no improvement for **patience** epochs; restores weights from best epoch.

```
for dcfg in ParameterGrid(dnn_grid):
    dnn = build_tfidf_dnn(
        input_dim=X_tr_dense.shape[1],
        num_classes=num_classes,
        dense1=dcfg['dense1'],
        dense2=dcfg['dense2'],
        drop=dcfg['drop'],
        lr=dcfg['lr']
    )
```

- **X_tr_dense.shape[1]** — Feature dimension (must match first Dense input).

```
dnn.fit(
    X_tr_dense, y_train_oh,
    validation_data=(X_va_dense, y_val_oh),
    epochs=dcfg['epochs'],
    batch_size=dcfg['batch_size'],
    verbose=1,
    callbacks=[early_stop]
)
```

- **y_train_oh** — One-hot training targets.
- **validation_data** — Tuple (X_val, y_val_oh) for loss monitoring.

```
val_pred = np.argmax(dnn.predict(X_va_dense, verbose=0), axis=1)
```

- **dnn.predict** — Probability matrix shape (n_val, num_classes).
- **np.argmax(..., axis=1)** — Class index with max probability per row.

```
val_f1 = f1_score(y_val, val_pred, average='macro')
add_tuning_log(prepare_name, 'TF-IDF', 'DeepNN', dcfg, val_f1)
```

- Logs validation score with **dcfg** (DNN hyperparameters).

```
if val_f1 > best_dnn_f1:
    best_dnn_f1 = val_f1
    best_dnn_model = dnn
```

- Keeps best Keras model reference.

Best DNN on test

```
test_pred_dnn = np.argmax(best_dnn_model.predict(X_te_dense, verbose=0), axis=1)
evaluate_and_store(y_test, test_pred_dnn, prepare_name, 'TF-IDF', 'DeepNN')
```

Mechanism of the whole cell

For each preprocessing: vectorize text → tune LR on sparse TF-IDF → test best LR → densify TF-IDF → tune MLP → test best MLP.

Role and importance in the notebook

Implements the **bag-of-words** + **classical ML** + **dense neural** track before sequence

models.